Architecture Design

Architecture Overview:

The system follows a cloud-based client–server architecture hosted entirely on Microsoft Azure. The client application (App) allows users to submit a webpage URL for fake news analysis and displays the credibility results.The app communicates with the backend through HTTPS requests and does not perform any AI processing or database access locally.The system shall use Azure Database for PostgreSQL as a managed cloud database to store all analysis data.

The database shall store structured information including analyzed URLs, credibility scores, AI summaries, extracted claims, sources, and user history. The database shall be accessed only by the Azure Functions backend through secure connection strings stored as environment variables; the client app shall not access the database directly. The backend shall receive webpage URLs from the client app via an Azure Functions API, extract the webpage content, and perform AI-based credibility analysis. The backend shall process and store the analysis results in Azure Database for PostgreSQL and return the credibility score and report to the client app.

Languages

Frontend :

React Native
Tailwind / CSS
Javascript

Backend :
Azure API
Azure Cloud Services : PostgreSQL

Frameworks
React Native

Deployment

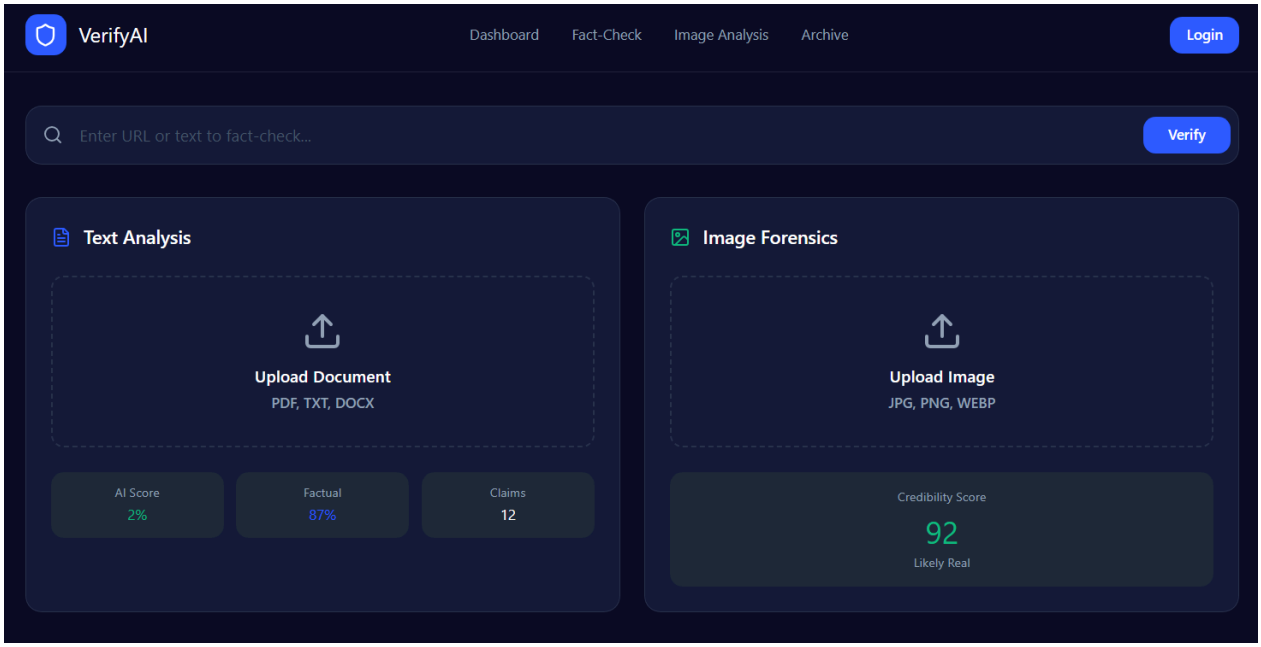
Hosting providers will include Microsoft Azure along with the following services to run our AI services:

- Azure Cloud Services
- Azure Language - Natural Language Processing
- Azure Document Intelligence - Text Extraction
- Azure Vision - Image Detection
  Azure Open AI - Language Learning Model
- Azure Database for PostgreSQL
- Azure Functions

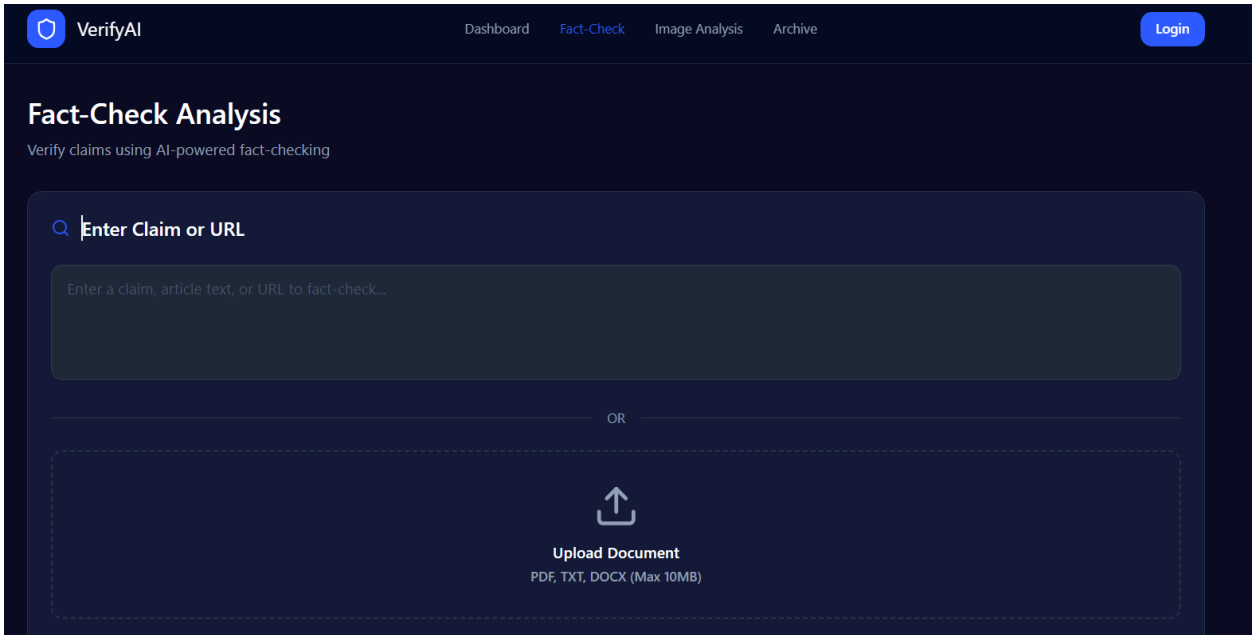
Since running a local instance of a language learning model is computationally demanding on local hardware, the application will instead utilize cloud-based AI services through Microsoft Azure cloud services. This will allow us to run our preferred language without the need of maintenance of a local server. Using Azure will also add to the capabilities of VerifyAI to leverage their many AI services to run scans towards a more accurate and comprehensive analysis.

What Type of Web App is VerifyAI

VerifyAI intends to be a single page application (SPA). This choice is by design to best facilitate the ease of use for the website as well as make the functionality for the app to be fast and efficient. This will be carried out by having the home page show the main content. This minimizes the need for nested menus and side tabs as they would be unnecessary but also add complexity and confusion to the user experience. At the very most, the user will have presented to them their login screen to sign in or sign up.

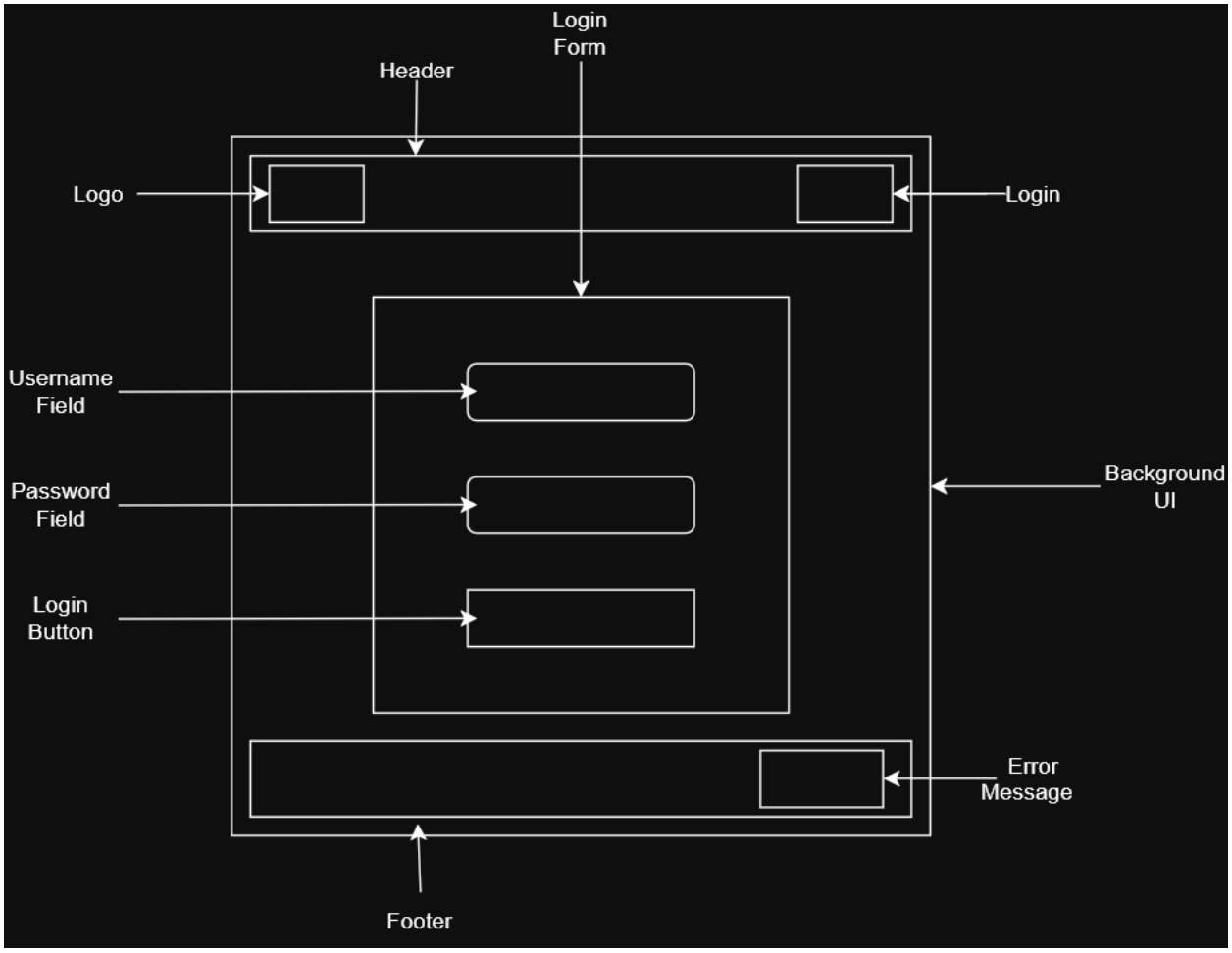


Layout of VerifyAI main homescreen



Layout of VerifyAI URL input analysis

Login View



Header - Navigation and Redirect

- Logo - Redirect to the home page
- Login - Direct user to the login page
- Background UI - Display main content, forms, and themes

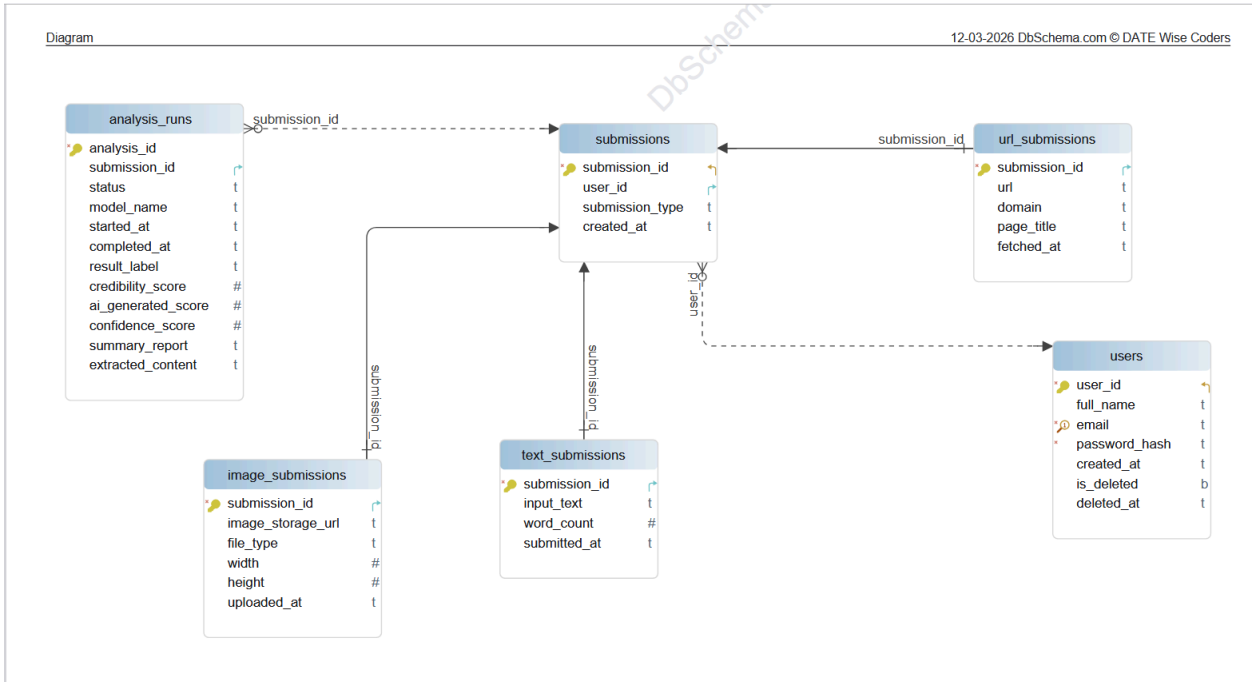
Login Form - Collect user credentials

- Username Field - Input for user identifier
- Password Field - Input for user credentials
- Login Button - Submit credentials to the authentication service

Footer

- Error Message - Display invalid credentials, lockouts, service interruptions

Database Schema



Link:
https://drive.google.com/file/d/1jy7msuZ746rzt4wYUlrXKjFnsorYAdC7/view?usp=sharing

Database Schema Documentation

This section describes the database schema used in the system. The database stores users, their submissions (text, images, or URLs), and the AI analysis results generated from those submissions. The relationships between the tables are shown in the database diagram above.

Users Table

The **users** table stores information about registered users in the system.

Attributes:

- **user_id (uuid)**
  Primary key that uniquely identifies each user.
- **full_name (varchar(255))**
  Stores the full name of the user.

- **email (varchar(255))**
  Stores the user's email address. This value must be unique.
- **password_hash (text)**
  Stores the encrypted password used for authentication.
- **created_at (timestamp)**
  Stores the date and time when the account was created.
- **is_deleted (boolean)**
  Indicates whether the user account has been deleted.
- **deleted_at (timestamp)**
  Stores the time when the account was deleted, if applicable.

Submissions Table

The **submissions** table stores basic information about all content submitted by users.

Attributes:

- **submission_id (uuid)**
  Primary key that uniquely identifies each submission.
- **user_id (uuid)**
  Foreign key referencing users.user_id.
- **submission_type (varchar(10))**
  Specifies the type of submission (URL, IMAGE, or TEXT).
- **created_at (timestamp)**
  Stores the time when the submission was created.

Text_Submissions Table

The **text_submissions** table stores details for submissions that contain text content.

Attributes:

- **submission_id (uuid)**
  Primary key and foreign key referencing submissions.submission_id.
- **input_text (text)**
  Stores the text submitted by the user.

- **word_count (integer)**
  Stores the number of words in the text.
- **submitted_at (timestamp)**
  Stores the date and time when the text was submitted.

Image_Submissions Table

The **image_submissions** table stores metadata about submitted images.

Attributes:

- **submission_id (uuid)**
  Primary key and foreign key referencing submissions.submission_id.
- **image_storage_url (text)**
  Stores the location or URL of the uploaded image.
- **file_type (varchar(10))**
  Stores the file format of the image.
- **width (integer)**
  Stores the width of the image in pixels.
- **height (integer)**
  Stores the height of the image in pixels.
- **uploaded_at (timestamp)**
  Stores the date and time when the image was uploaded.

URL_Submissions Table

The **url_submissions** table stores information about submissions containing URLs.

Attributes:

- **submission_id (uuid)**
  Primary key and foreign key referencing submissions.submission_id.
- **url (text)**
  Stores the submitted website URL.
- **domain (varchar(255))**
  Stores the domain extracted from the URL.

- **page_title (text)**
  Stores the title of the web page.
- **fetched_at (timestamp)**
  Stores the date and time when the webpage data was retrieved.

Analysis_Runs Table

The **analysis_runs** table stores AI analysis results for submissions.

Attributes:

- **analysis_id (uuid)**
  Primary key that uniquely identifies each analysis run.
- **submission_id (uuid)**
  Foreign key referencing submissions.submission_id.
- **status (varchar(20))**
  Indicates the current status of the analysis (queued, running, completed, failed).
- **model_name (varchar(255))**
  Stores the name of the AI model used.
- **started_at (timestamp)**
  Stores when the analysis started.
- **completed_at (timestamp)**
  Stores when the analysis finished.
- **result_label (varchar(50))**
  Stores the final classification result.
- **credibility_score (integer)**
  Stores the credibility score of the content.
- **ai_generated_score (integer)**
  Stores the score estimating whether the content was AI generated.
- **confidence_score (integer)**
  Stores the confidence level of the analysis.
- **summary_report (text)**
  Stores a summary explaining the analysis results.
- **extracted_content (text)**
  Stores content extracted from the submission during analysis.

Table Relationships

The tables in this schema are connected through foreign key relationships.

users.user_id → submissions.user_id
Each submission belongs to a specific user.

submissions.submission_id → text_submissions.submission_id
Links text submissions to their base submission.

submissions.submission_id → image_submissions.submission_id
Links image submissions to their base submission.

submissions.submission_id → url_submissions.submission_id
Links URL submissions to their base submission.

submissions.submission_id → analysis_runs.submission_id
Links analysis results to the submission being analyzed.

This structure allows the system to store multiple types of submissions while keeping the core information centralized in the submissions table.